

MYSQL TRIGGERS

Practical Examples and Student Exercises using MySQL Workbench

What is a Trigger in MySQL?

A Trigger is a database object that automatically executes when a specified event occurs on a table.

Note: You don't call a trigger manually. MySQL automatically executes it when INSERT, UPDATE, or DELETE happens.

Why do we use Triggers?

Triggers are mainly used for:

- **Auditing** – record who/what was changed
- **Validation** – prevent invalid data
- **Automatic calculations**
- **Maintaining related/summary data**
- **Logging INSERT, UPDATE and DELETE operations**
- **Enforcing business rules**

Types of MySQL Triggers

MySQL triggers are classified based on **when** they execute and **what event** activates them.

Timing

1. BEFORE
2. AFTER

Events

1. INSERT
2. UPDATE
3. DELETE

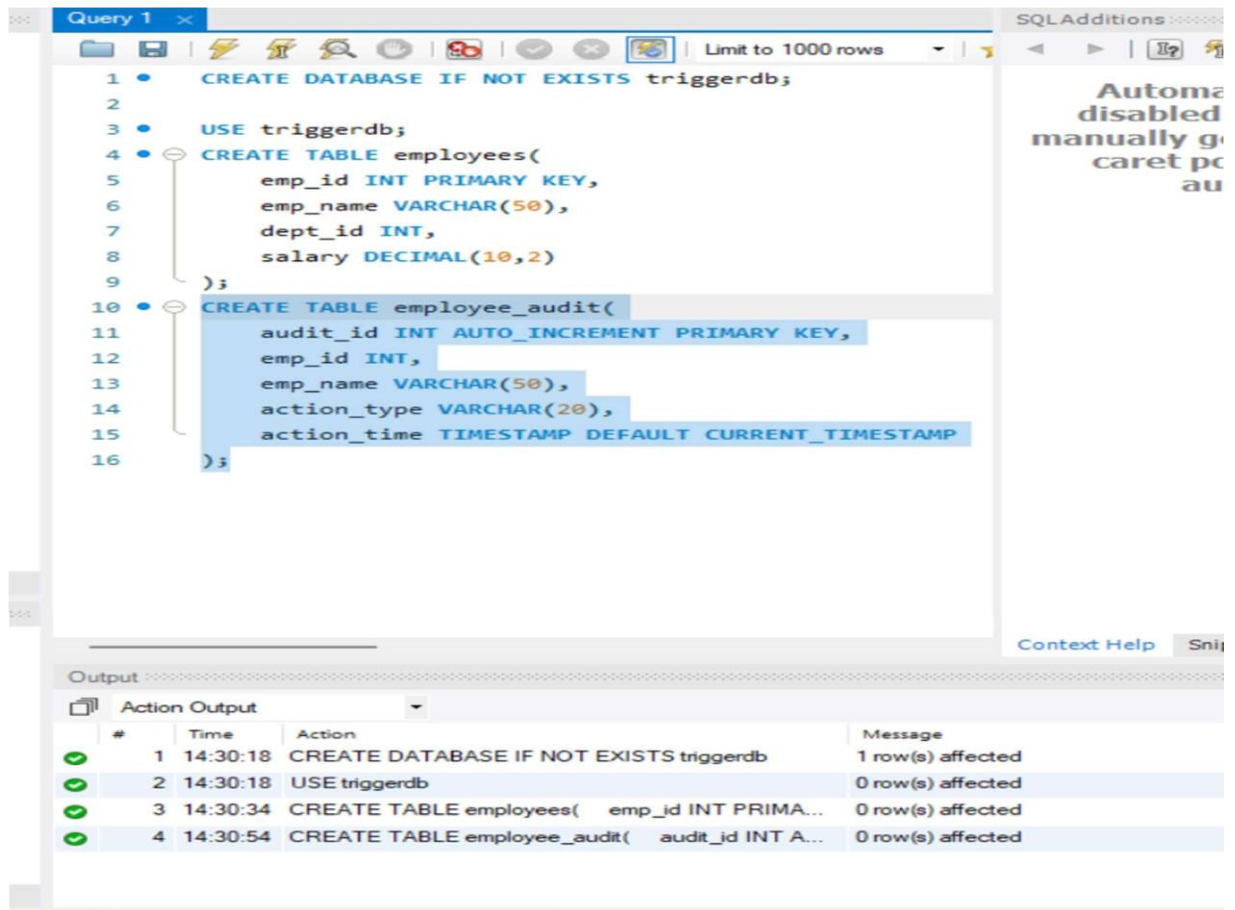
- **First Trigger Example – AFTER INSERT**

Create an employees table:

```
CREATE TABLE employees(  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    dept_id INT,  
    salary DECIMAL(10,2)  
);
```

Create an audit table:

```
CREATE TABLE employee_audit(  
    audit_id INT AUTO_INCREMENT PRIMARY KEY,  
    emp_id INT,  
    emp_name VARCHAR(50),  
    action_type VARCHAR(20),  
    action_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```



Now create the trigger:

DELIMITER \$\$

CREATE TRIGGER after_employee_insert

AFTER INSERT ON employees

FOR EACH ROW

BEGIN

INSERT INTO employee_audit

(emp_id, emp_name, action_type)

VALUES

(NEW.emp_id, NEW.emp_name, 'INSERT');

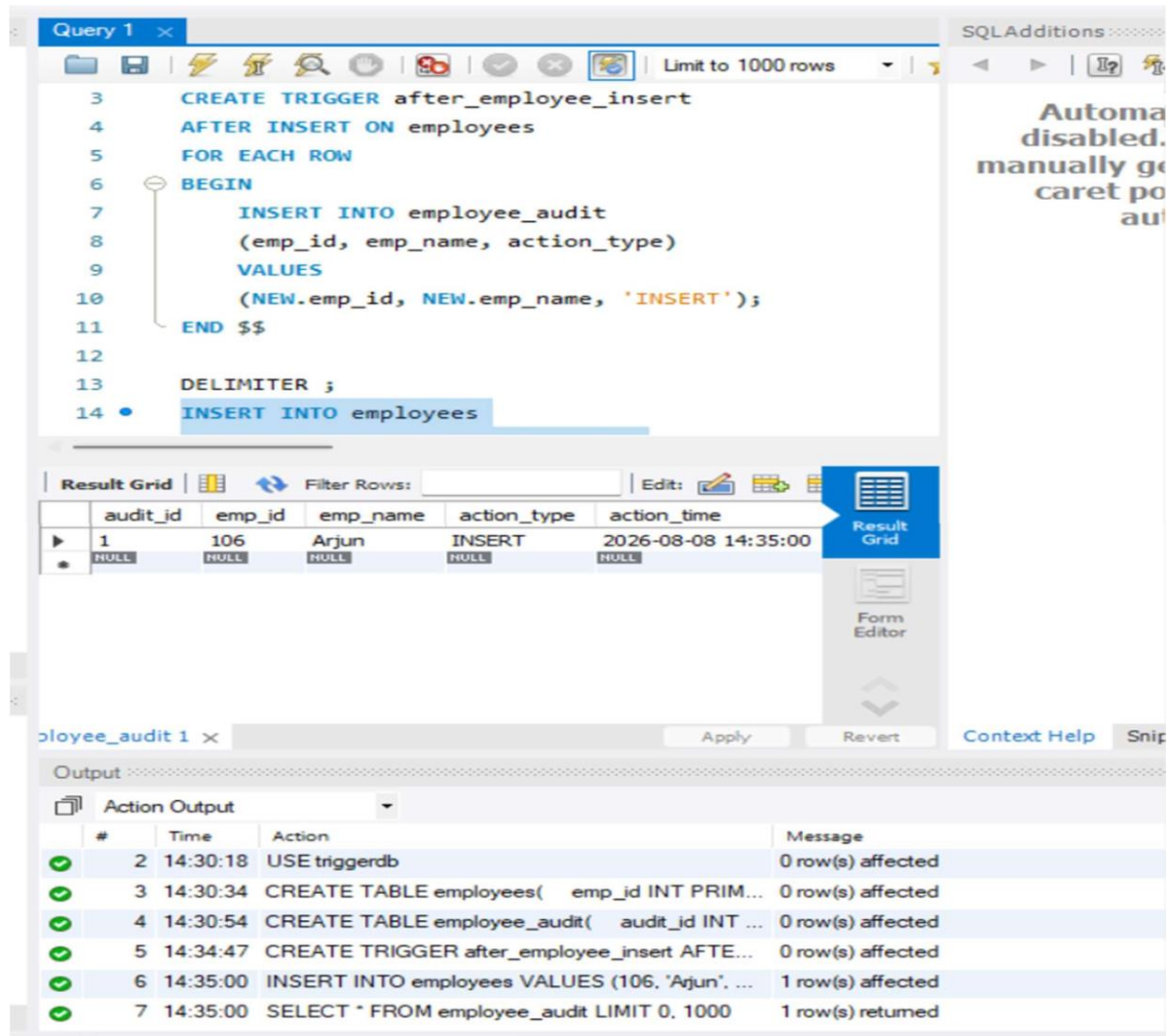
END \$\$

DELIMITER ;

Now insert an employee:

INSERT INTO employees

VALUES (106, 'Arjun', 1, 52000);



The screenshot displays a SQL IDE interface. The main editor window shows the following SQL code:

```
3 CREATE TRIGGER after_employee_insert
4 AFTER INSERT ON employees
5 FOR EACH ROW
6 BEGIN
7     INSERT INTO employee_audit
8     (emp_id, emp_name, action_type)
9     VALUES
10    (NEW.emp_id, NEW.emp_name, 'INSERT');
11 END $$
12
13 DELIMITER ;
14 INSERT INTO employees
```

Below the editor, the 'Result Grid' shows the execution results of the SQL statements:

audit_id	emp_id	emp_name	action_type	action_time
1	106	Arjun	INSERT	2026-08-08 14:35:00
NULL	NULL	NULL	NULL	NULL

The 'Output' pane at the bottom shows the sequence of SQL actions and their results:

#	Time	Action	Message
2	14:30:18	USE triggerdb	0 row(s) affected
3	14:30:34	CREATE TABLE employees(emp_id INT PRIM...	0 row(s) affected
4	14:30:54	CREATE TABLE employee_audit(audit_id INT ...	0 row(s) affected
5	14:34:47	CREATE TRIGGER after_employee_insert AFTE...	0 row(s) affected
6	14:35:00	INSERT INTO employees VALUES (106, 'Arjun', ...	1 row(s) affected
7	14:35:00	SELECT * FROM employee_audit LIMIT 0, 1000	1 row(s) returned

> You **didn't call the trigger**. It automatically executes.

SELECT * FROM employee_audit;

Output:

106 Arjun INSERT

- **Second Trigger Example – AFTER DELETE**

Suppose Employee deleted:

```
DELETE FROM employees
```

```
WHERE emp_id = 106;
```

We can automatically record that deletion.

```
DELIMITER $$
```

```
CREATE TRIGGER after_employee_delete
```

```
AFTER DELETE ON employees
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    INSERT INTO employee_audit
```

```
        (emp_id, emp_name, action_type)
```

```
VALUES
```

```
    (OLD.emp_id, OLD.emp_name, 'DELETE');
```

```
END $$
```

Query 1 x

SQLAdditions

Limit to 1000 rows

```

9      INSERT INTO employee_audit
10     (emp_id, emp_name, action_type)
11     VALUES
12     (OLD.emp_id, OLD.emp_name, 'DELETE');
13     END $$
14
15     DELIMITER ;
16 •   INSERT INTO employees
17     VALUES (106, 'Arjun', 1, 52000);
18 •   DELETE FROM employees
19     WHERE emp_id = 106;
20 •   SELECT * FROM employee_audit;

```

Result Grid

	audit_id	emp_id	emp_name	action_type	action_time
▶	1	106	Arjun	INSERT	2026-08-08 14:35:00
	2	106	Arjun	INSERT	2026-08-08 14:46:38
	3	106	Arjun	DELETE	2026-08-08 14:46:50
★	NULL	NULL	NULL	NULL	NULL

Form Editor

employee_audit 2 x

Apply Revert Context Help Snip

Output

Action Output

#	Time	Action	Message
✖	10 14:45:39	CREATE TRIGGER after_employee_delete AFTE...	Error Code: 1064. You have an error in y
⚠	11 14:46:22	DROP TRIGGER IF EXISTS after_employee_del...	0 row(s) affected, 1 warning(s): 1360 Trig
✓	12 14:46:22	CREATE TRIGGER after_employee_delete AFTE...	0 row(s) affected
✓	13 14:46:38	INSERT INTO employees VALUES (106, 'Arjun', ...	1 row(s) affected
✓	14 14:46:50	DELETE FROM employees WHERE emp_id = 106	1 row(s) affected
✓	15 14:48:36	SELECT * FROM employee_audit LIMIT 0, 1000	3 row(s) returned

DELIMITER ;

• Third Trigger Example--AFTER UPDATE Trigger:

UPDATE employees

SET salary = 60000

WHERE emp_id = 101;

We can record the update.

DELIMITER \$\$

CREATE TRIGGER after_employee_update

```
AFTER UPDATE ON employees  
  
FOR EACH ROW  
  
BEGIN  
  
    INSERT INTO employee_audit  
  
        (emp_id, emp_name, action_type)  
  
VALUES  
  
    (NEW.emp_id, NEW.emp_name, 'UPDATE');  
  
END $$  
  
DELIMITER ;
```


Query 1 x SQLAdditions

Limit to 1000 rows

```

8 BEGIN
9     INSERT INTO employee_audit
10    (emp_id, emp_name, action_type)
11    VALUES
12    (NEW.emp_id, NEW.emp_name, 'UPDATE');
13 END $$
14
15 DELIMITER ;
16 • UPDATE employees
17   SET salary = 60000
18   WHERE emp_id = 101;
19 • SELECT * FROM employee_audit;

```

Result Grid

	audit_id	emp_id	emp_name	action_type	action_time
▶	1	106	Arjun	INSERT	2026-08-08 14:35:00
	2	106	Arjun	INSERT	2026-08-08 14:46:38
	3	106	Arjun	DELETE	2026-08-08 14:46:50
•	NULL	NULL	NULL	NULL	NULL

Form Editor

employee_audit 3 x Apply Revert Context Help Snip

Output

Action Output

#	Time	Action	Message
✓ 14	14:46:50	DELETE FROM employees WHERE emp_id = 106	1 row(s) affected
✓ 15	14:48:36	SELECT * FROM employee_audit LIMIT 0, 1000	3 row(s) returned
⚠ 16	14:51:35	DROP TRIGGER IF EXISTS after_employee_upd...	0 row(s) affected, 1 warning(s): 1360 Trig
✓ 17	14:51:35	CREATE TRIGGER after_employee_update AFT...	0 row(s) affected
✓ 18	14:52:23	UPDATE employees SET salary = 60000 WHER...	0 row(s) affected Rows matched: 0 Cha
✓ 19	14:52:35	SELECT * FROM employee_audit LIMIT 0, 1000	3 row(s) returned

- **Fourth Trigger Example--BEFORE INSERT Trigger:**

Employee salary cannot be less than ₹20,000.

```
DELIMITER $$

CREATE TRIGGER check_salary

BEFORE INSERT ON employees

FOR EACH ROW

BEGIN

    IF NEW.salary < 20000 THEN

        SIGNAL SQLSTATE '45000'

        SET MESSAGE_TEXT = 'Salary cannot be less than 20000';

    END IF;

END $$

DELIMITER ;
```

The screenshot shows a SQL query editor window titled "Query 1". The query script is as follows:

```
1 DROP TRIGGER IF EXISTS check_salary;
2
3 DELIMITER $$
4
5 CREATE TRIGGER check_salary
6 BEFORE INSERT ON employees
7 FOR EACH ROW
8 BEGIN
9     IF NEW.salary < 20000 THEN
10         SIGNAL SQLSTATE '45000'
11         SET MESSAGE_TEXT = 'Salary cannot be less than 20000';
12     END IF;
13 END $$
14
15 DELIMITER ;
```

Below the query editor is the "Output" pane, which displays the "Action Output" of the executed queries:

#	Time	Action	Message
16	14:51:35	DROP TRIGGER IF EXISTS after_employee_upd...	0 row(s) affected, 1 warning(s): 1360 Trig
17	14:51:35	CREATE TRIGGER after_employee_update AFT...	0 row(s) affected
18	14:52:23	UPDATE employees SET salary = 60000 WHER...	0 row(s) affected Rows matched: 0 Cha
19	14:52:35	SELECT * FROM employee_audit LIMIT 0, 1000	3 row(s) returned
20	14:53:17	DROP TRIGGER IF EXISTS check_salary	0 row(s) affected, 1 warning(s): 1360 Trig
21	14:53:17	CREATE TRIGGER check_salary BEFORE INSE...	0 row(s) affected

Now, run this Query

INSERT INTO employees

VALUES (107, 'Ravi', 1, 15000);

The screenshot shows a MySQL IDE window with a SQL query editor and an output pane. The query editor contains the following SQL code:

```
1 • DROP TRIGGER IF EXISTS check_salary;
2
3 DELIMITER $$
4
5 CREATE TRIGGER check_salary
6 BEFORE INSERT ON employees
7 FOR EACH ROW
8 BEGIN
9     IF NEW.salary < 20000 THEN
10         SIGNAL SQLSTATE '45000'
11         SET MESSAGE_TEXT = 'Salary cannot be less than 20000';
12     END IF;
13 END $$
14
15 DELIMITER ;
16 • INSERT INTO employees
17 VALUES (107, 'Ravi', 1, 15000);
```

The output pane shows the results of the query execution:

#	Time	Action	Message
✓ 17	14:51:35	CREATE TRIGGER after_employee_update AFT...	0 row(s) affected
✓ 18	14:52:23	UPDATE employees SET salary = 60000 WHER...	0 row(s) affected Rows matched: 0 Cha
✓ 19	14:52:35	SELECT * FROM employee_audit LIMIT 0, 1000	3 row(s) returned
⚠ 20	14:53:17	DROP TRIGGER IF EXISTS check_salary	0 row(s) affected, 1 warning(s): 1360 Trig
✓ 21	14:53:17	CREATE TRIGGER check_salary BEFORE INSE...	0 row(s) affected
✗ 22	14:53:46	INSERT INTO employees VALUES (107, 'Ravi', 1...	Error Code: 1644. Salary cannot be less

MySQL will reject it and display:

Salary cannot be less than 20000

But, If you run this Query it will succeed as salary is greater than 20000

INSERT INTO employees VALUES (108, 'Meena', 1, 30000);

The screenshot displays the SQL Developer interface. The 'Query 1' window contains the following SQL code:

```
1 DROP TRIGGER IF EXISTS check_salary;
2
3 DELIMITER $$
4
5 CREATE TRIGGER check_salary
6 BEFORE INSERT ON employees
7 FOR EACH ROW
8 BEGIN
9     IF NEW.salary < 20000 THEN
10         SIGNAL SQLSTATE '45000'
11         SET MESSAGE_TEXT = 'Salary cannot be less than 20000';
12     END IF;
13 END $$
14
15 DELIMITER ;
16 INSERT INTO employees
17 VALUES (107, 'Ravi', 1, 15000);
18 INSERT INTO employees
19 VALUES (108, 'Meena', 1, 30000);
```

The 'Output' window shows the execution results:

#	Time	Action	Message
✓ 18	14:52:23	UPDATE employees SET salary = 60000 WHERE...	0 row(s) affected Rows matched: 0 Cha
✓ 19	14:52:35	SELECT * FROM employee_audit LIMIT 0, 1000	3 row(s) returned
⚠ 20	14:53:17	DROP TRIGGER IF EXISTS check_salary	0 row(s) affected, 1 warning(s): 1360 Trig
✓ 21	14:53:17	CREATE TRIGGER check_salary BEFORE INSE...	0 row(s) affected
✗ 22	14:53:46	INSERT INTO employees VALUES (107, 'Ravi', 1...	Error Code: 1644. Salary cannot be less
✓ 23	14:54:14	INSERT INTO employees VALUES (108, 'Meena'...	1 row(s) affected

- **Fifth Trigger Example--BEFORE UPDATE Trigger:**

Suppose the company does not allow salary reduction.

DELIMITER \$\$

CREATE TRIGGER prevent_salary_reduction

BEFORE UPDATE ON employees

FOR EACH ROW

BEGIN

IF NEW.salary < OLD.salary THEN

SIGNAL SQLSTATE '45000'

SET MESSAGE_TEXT = 'Salary reduction is not allowed';

END IF;

END \$\$

DELIMITER ;

The screenshot displays the SQL Server Enterprise Manager interface. The main window shows a script for creating a trigger named 'prevent_salary_reduction' on the 'employees' table. The script includes a 'DROP TRIGGER IF EXISTS' statement, followed by 'DELIMITER \$\$', then the 'CREATE TRIGGER' statement with 'BEFORE UPDATE ON employees' and 'FOR EACH ROW' clauses. The trigger body contains an 'IF NEW.salary < OLD.salary THEN' condition, which triggers a 'SIGNAL SQLSTATE '45000'' and sets 'MESSAGE_TEXT = 'Salary reduction is not allowed''. The script ends with 'END IF;', 'END \$\$', and 'DELIMITER ;'.

The 'Output' pane at the bottom shows the execution results of the script. It lists five actions with their respective times and messages:

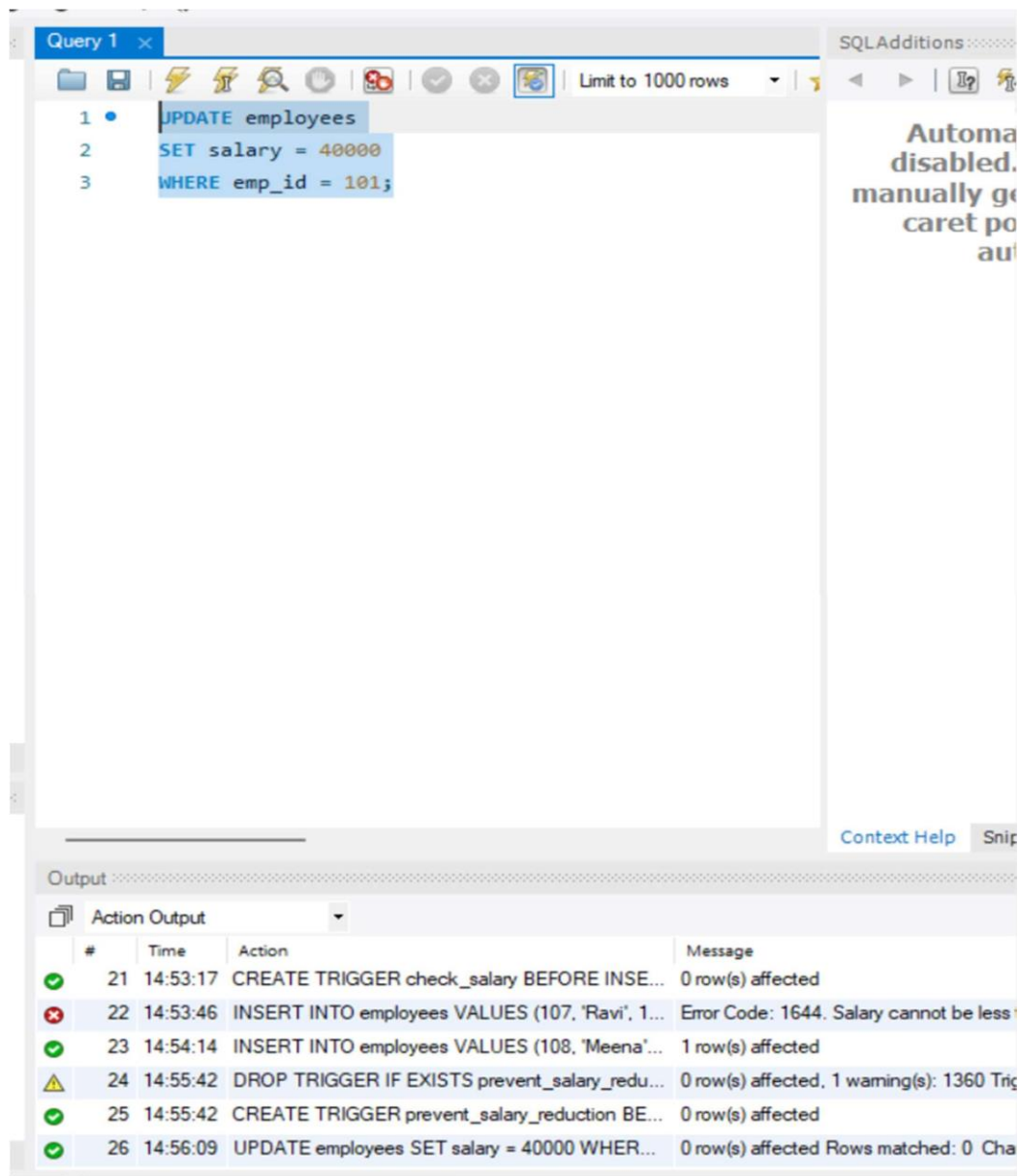
#	Time	Action	Message
20	14:53:17	DROP TRIGGER IF EXISTS check_salary	0 row(s) affected, 1 warning(s): 1360 Trg
21	14:53:17	CREATE TRIGGER check_salary BEFORE INSE...	0 row(s) affected
22	14:53:46	INSERT INTO employees VALUES (107, 'Ravi', 1...	Error Code: 1644, Salary cannot be less
23	14:54:14	INSERT INTO employees VALUES (108, 'Meena'...	1 row(s) affected
24	14:55:42	DROP TRIGGER IF EXISTS prevent_salary_redu...	0 row(s) affected, 1 warning(s): 1360 Trg
25	14:55:42	CREATE TRIGGER prevent_salary_reduction BE...	0 row(s) affected

Now, run this Update Query

UPDATE employees

SET salary = 40000

WHERE emp_id = 101;



If the old salary was ₹50,000, the trigger rejects the update.

But, if you run the below update query, it succeeds as you are not reducing salary

UPDATE employees

SET salary = 60000

WHERE emp_id = 101;

The screenshot shows a SQL query editor with two queries. The first query (lines 1-3) updates the salary of employee 101 to 40000. The second query (lines 4-6) updates the salary of employee 101 to 60000. The output window shows the execution results of these queries.

Query 1

```
1 • UPDATE employees
2   SET salary = 40000
3   WHERE emp_id = 101;
4 • UPDATE employees
5   SET salary = 60000
6   WHERE emp_id = 101;
```

Output

#	Time	Action	Message
22	14:53:46	INSERT INTO employees VALUES (107, 'Ravi', 1...	Error Code: 1644. Salary cannot be less
23	14:54:14	INSERT INTO employees VALUES (108, 'Meena'...	1 row(s) affected
24	14:55:42	DROP TRIGGER IF EXISTS prevent_salary_redu...	0 row(s) affected, 1 warning(s): 1360 Trg
25	14:55:42	CREATE TRIGGER prevent_salary_reduction BE...	0 row(s) affected
26	14:56:09	UPDATE employees SET salary = 40000 WHER...	0 row(s) affected Rows matched: 0 Cha
27	14:56:40	UPDATE employees SET salary = 60000 WHER...	0 row(s) affected Rows matched: 0 Cha

Here, after salary update

OLD.salary → 50000

NEW.salary → 60000

